

```
In [97]: # import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
In [98]: # import database / data
conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table'
    """

tables = pd.read_sql(sql_query , conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}" , conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe : df_{table_name}")
    print(tables)

conn.close()
print("hello ")
```

Created dataframe : df_subscription

```
      name
0  subscription
1      support
2    customers
```

Created dataframe : df_support

```
      name
0  subscription
1      support
2    customers
```

Created dataframe : df_customers

```
      name
0  subscription
1      support
2    customers
```

hello

```
In [99]: conn = sqlite3.connect("customer_churn.db")
print("Database connected")
```

Database connected

```
In [100... tables = pd.read_sql("""
SELECT name
FROM sqlite_master
WHERE type='table'
""",conn)
print(tables)
```

```

    name
0  subscription
1      support
2    customers

```

```

In [101... import pandas as pd

df = pd.read_csv("customer_churn.csv")

print(df)

```

	customerid	name	country	state	gender	dob \
0	0002-ORFBO	keshav	India	Maharashtra	Male	4/12/1982
1	0003-MKNFE	raghav	India	Karnataka	Male	11/23/1995
2	0004-TLHLJ	lalita	India	Delhi	Female	2/15/1978
3	0011-IGKFF	mohan	India	Nagaland	Male	8/30/2001
4	0013-EXCHZ	mira	India	Delhi	Female	5/5/1990
5	0013-MHZWF	durga	NaN	Delhi	Women	12/10/1988
6	0013-SMEOE	mina	India	Meghalaya	Female	9/21/1976
7	0014-BMAQU	madan	India	Rajasthan	Male	3/14/1999
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	7/7/1985
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	10/29/1993
10	0017-DINOC	shiva	India	Maharashtra	Men	1/22/1997
11	0017-IUDMW	rangadevi	India	Karnataka	Female	6/18/1981
12	0018-NYROU	chitra	NaN	Telangana	Female	12/1/2004
13	0019-EFAEP	raju	India	Meghalaya	Female	4/25/1992
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	11/11/1979
15	0020-INWCK	parvati	India	Delhi	Female	2/28/1986
16	0020-JDNXP	rikim	India	Meghalaya	Female	8/19/1994
17	0021-IKXGC	vishakha	India	Rajasthan	Female	9/2/2000
18	0022-TCJCI	raghvendra	India	Telangana	Male	12/30/1983
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	5/14/1991
20	0023-UYUPN	sudevi	India	Maharashtra	Women	10/6/1977

	interests	pincode
0	travel	NaN
1	NaN	NaN
2	movie	NaN
3	NaN	NaN
4	drama	NaN
5	NaN	NaN
6	NaN	NaN
7	NaN	NaN
8	NaN	NaN
9	NaN	NaN
10	NaN	NaN
11	NaN	NaN
12	NaN	NaN
13	NaN	NaN
14	NaN	NaN
15	job	NaN
16	NaN	NaN
17	NaN	NaN
18	NaN	NaN
19	NaN	NaN
20	NaN	NaN

```

In [102... import sqlite3
import pandas as pd

```

```

conn = sqlite3.connect("customer_churn.db")

query = """
SELECT name
FROM sqlite_master
WHERE type='table';
"""

tables = pd.read_sql_query(query, conn)

print(tables)

```

```

      name
0  subscription
1      support
2    customers

```

```

In [103... import pandas as pd
import sqlite3

# CSV read karo
df = pd.read_csv("customer_churn.csv")

# Database create/connect karo
conn = sqlite3.connect("customer_churn.db")

# CSV data ko SQLite table me save karo
df.to_sql("customers", conn, if_exists="replace", index=False)

print("Database created successfully!")

```

Database created successfully!

```

In [104... conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table'
    """

tables = pd.read_sql(sql_query , conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}" , conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe : df_{table_name}")

conn.close()

```

```

Created dataframe : df_subscription
Created dataframe : df_support
Created dataframe : df_customers

```

```

In [105... print(df_customers.head())
print(df_subscription.head())
print(df_support.head())

```

	customerid	name	country	state	gender	dob	interests	\
0	0002-ORFBO	keshav	India	Maharashtra	Male	4/12/1982	travel	
1	0003-MKNFE	raghav	India	Karnataka	Male	11/23/1995	None	
2	0004-TLHLJ	lalita	India	Delhi	Female	2/15/1978	movie	
3	0011-IGKFF	mohan	India	Nagaland	Male	8/30/2001	None	
4	0013-EXCHZ	mira	India	Delhi	Female	5/5/1990	drama	

pincode

0	None
1	None
2	None
3	None
4	None

	customerid	subscription_start_date	subscription_type	renewal_date	\
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	

	plan_type	contract_type	cancellation_date	cancellation_reason	\
0	Standard	Annual	None	None	
1	Premium	Annual	2024-09-10	Switched to competitor	
2	Basic	Monthly	None	None	
3	Premium	Annual	None	None	
4	Standard	Monthly	2024-02-28	Too expensive	

	monthly_charges	cltv	churn_score
0	13.99	627	12
1	12.99	1150	91
2	6.99	210	34
3	22.99	1725	8
4	13.99	195	88

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	8/28/2024	N	60	None	service issue
1	0003-MKNFE	8/28/2024	Y	10	None	demaned refund
2	0013-EXCHZ	1/20/2024	Y	20	None	None
3	0013-MHZWF	3/18/2025	N	90	None	guidance to renew
4	0013-SMEOE	11/1/2024	N	30	None	None

In [106...

```
# print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name:{table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query , conn)
    print("columns:")
    print(columns['name'].tolist())

# close connection
conn.close()
```

Table Name:subscription

columns:

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']
```

Table Name:support

columns:

```
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

Table Name:customers

columns:

```
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']
```

In [107...

```
df_customers.head()
# a . rename col -name
# b . drop columns - interest and pincode
# c . change data type dob
# d . data standardization - gender
# e. fix missing values - country
```

Out[107...

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	4/12/1982	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	11/23/1995	None	None
2	0004-TLHLJ	lalita	India	Delhi	Female	2/15/1978	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	8/30/2001	None	None
4	0013-EXCHZ	mira	India	Delhi	Female	5/5/1990	drama	None

In [108...

```
# a . rename col - name
df_customers.rename(columns = {'name' : 'customer_name'} , inplace = True )
df_customers
```

Out[108...

	customerid	customer_name	country	state	gender	dob	interests	
0	0002-ORFBO	keshav	India	Maharashtra	Male	4/12/1982	travel	
1	0003-MKNFE	raghav	India	Karnataka	Male	11/23/1995	None	
2	0004-TLHLJ	lalita	India	Delhi	Female	2/15/1978	movie	
3	0011-IGKFF	mohan	India	Nagaland	Male	8/30/2001	None	
4	0013-EXCHZ	mira	India	Delhi	Female	5/5/1990	drama	
5	0013-MHZWF	durga	None	Delhi	Women	12/10/1988	None	
6	0013-SMEOE	mina	India	Meghalaya	Female	9/21/1976	None	
7	0014-BMAQU	madan	India	Rajasthan	Male	3/14/1999	None	
8	0015-UOCOJ	maya	None	Kathmandu	Women	7/7/1985	None	
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	10/29/1993	None	
10	0017-DINOC	shiva	India	Maharashtra	Men	1/22/1997	None	
11	0017-IUDMW	rangadevi	India	Karnataka	Female	6/18/1981	None	
12	0018-NYROU	chitra	None	Telangana	Female	12/1/2004	None	
13	0019-EFAEP	raju	India	Meghalaya	Female	4/25/1992	None	
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	11/11/1979	None	
15	0020-INWCK	parvati	India	Delhi	Female	2/28/1986	job	
16	0020-JDNXP	rikim	India	Meghalaya	Female	8/19/1994	None	
17	0021-IKXGC	vishakha	India	Rajasthan	Female	9/2/2000	None	
18	0022-TCJCI	raghvendra	India	Telangana	Male	12/30/1983	None	
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	5/14/1991	None	
20	0023-UYUPN	sudevi	India	Maharashtra	Women	10/6/1977	None	

In [109...

```
df_customers.drop(df_customers.columns[-2:], axis = 1, inplace = True)
df_customers
```

Out[109...

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	4/12/1982
1	0003-MKNFE	raghav	India	Karnataka	Male	11/23/1995
2	0004-TLHLJ	lalita	India	Delhi	Female	2/15/1978
3	0011-IGKFF	mohan	India	Nagaland	Male	8/30/2001
4	0013-EXCHZ	mira	India	Delhi	Female	5/5/1990
5	0013-MHZWF	durga	None	Delhi	Women	12/10/1988
6	0013-SMEOE	mina	India	Meghalaya	Female	9/21/1976
7	0014-BMAQU	madan	India	Rajasthan	Male	3/14/1999
8	0015-UOCOJ	maya	None	Kathmandu	Women	7/7/1985
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	10/29/1993
10	0017-DINOC	shiva	India	Maharashtra	Men	1/22/1997
11	0017-IUDMW	rangadevi	India	Karnataka	Female	6/18/1981
12	0018-NYROU	chitra	None	Telangana	Female	12/1/2004
13	0019-EFAEP	raju	India	Meghalaya	Female	4/25/1992
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	11/11/1979
15	0020-INWCK	parvati	India	Delhi	Female	2/28/1986
16	0020-JDNXP	rikim	India	Meghalaya	Female	8/19/1994
17	0021-IKXGC	vishakha	India	Rajasthan	Female	9/2/2000
18	0022-TCJCI	raghvendra	India	Telangana	Male	12/30/1983
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	5/14/1991
20	0023-UYUPN	sudevi	India	Maharashtra	Women	10/6/1977

In [110...

```
df_customers['dob'] = df_customers['dob'].str.replace('/', '-')
df_customers
```

Out[110...

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	4-12-1982
1	0003-MKNFE	raghav	India	Karnataka	Male	11-23-1995
2	0004-TLHLJ	lalita	India	Delhi	Female	2-15-1978
3	0011-IGKFF	mohan	India	Nagaland	Male	8-30-2001
4	0013-EXCHZ	mira	India	Delhi	Female	5-5-1990
5	0013-MHZWF	durga	None	Delhi	Women	12-10-1988
6	0013-SMEOE	mina	India	Meghalaya	Female	9-21-1976
7	0014-BMAQU	madan	India	Rajasthan	Male	3-14-1999
8	0015-UOCOJ	maya	None	Kathmandu	Women	7-7-1985
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	10-29-1993
10	0017-DINOC	shiva	India	Maharashtra	Men	1-22-1997
11	0017-IUDMW	rangadevi	India	Karnataka	Female	6-18-1981
12	0018-NYROU	chitra	None	Telangana	Female	12-1-2004
13	0019-EFAEP	raju	India	Meghalaya	Female	4-25-1992
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	11-11-1979
15	0020-INWCK	parvati	India	Delhi	Female	2-28-1986
16	0020-JDNXP	rikim	India	Meghalaya	Female	8-19-1994
17	0021-IKXGC	vishakha	India	Rajasthan	Female	9-2-2000
18	0022-TCJCI	raghvendra	India	Telangana	Male	12-30-1983
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	5-14-1991
20	0023-UYUPN	sudevi	India	Maharashtra	Women	10-6-1977

In [111...

```
df_customers['gender'] = df_customers['gender'].replace({
    'Men': 'Male',
    'Women': 'Female'
})
df_customers
```


Out[111...

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	4-12-1982
1	0003-MKNFE	raghav	India	Karnataka	Male	11-23-1995
2	0004-TLHLJ	lalita	India	Delhi	Female	2-15-1978
3	0011-IGKFF	mohan	India	Nagaland	Male	8-30-2001
4	0013-EXCHZ	mira	India	Delhi	Female	5-5-1990
5	0013-MHZWF	durga	None	Delhi	Female	12-10-1988
6	0013-SMEOE	mina	India	Meghalaya	Female	9-21-1976
7	0014-BMAQU	madan	India	Rajasthan	Male	3-14-1999
8	0015-UOCOJ	maya	None	Kathmandu	Female	7-7-1985
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	10-29-1993
10	0017-DINOC	shiva	India	Maharashtra	Male	1-22-1997
11	0017-IUDMW	rangadevi	India	Karnataka	Female	6-18-1981
12	0018-NYROU	chitra	None	Telangana	Female	12-1-2004
13	0019-EFAEP	raju	India	Meghalaya	Female	4-25-1992
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	11-11-1979
15	0020-INWCK	parvati	India	Delhi	Female	2-28-1986
16	0020-JDNXP	rikim	India	Meghalaya	Female	8-19-1994
17	0021-IKXGC	vishakha	India	Rajasthan	Female	9-2-2000
18	0022-TCJCI	raghvendra	India	Telangana	Male	12-30-1983
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	5-14-1991
20	0023-UYUPN	sudevi	India	Maharashtra	Female	10-6-1977

In [112...

```
# e. fix missing values - country
df_customers['country'].isna()
df_customers
```

Out[112...

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	4-12-1982
1	0003-MKNFE	raghav	India	Karnataka	Male	11-23-1995
2	0004-TLHLJ	lalita	India	Delhi	Female	2-15-1978
3	0011-IGKFF	mohan	India	Nagaland	Male	8-30-2001
4	0013-EXCHZ	mira	India	Delhi	Female	5-5-1990
5	0013-MHZWF	durga	None	Delhi	Female	12-10-1988
6	0013-SMEOE	mina	India	Meghalaya	Female	9-21-1976
7	0014-BMAQU	madan	India	Rajasthan	Male	3-14-1999
8	0015-UOCOJ	maya	None	Kathmandu	Female	7-7-1985
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	10-29-1993
10	0017-DINOC	shiva	India	Maharashtra	Male	1-22-1997
11	0017-IUDMW	rangadevi	India	Karnataka	Female	6-18-1981
12	0018-NYROU	chitra	None	Telangana	Female	12-1-2004
13	0019-EFAEP	raju	India	Meghalaya	Female	4-25-1992
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	11-11-1979
15	0020-INWCK	parvati	India	Delhi	Female	2-28-1986
16	0020-JDNXP	rikim	India	Meghalaya	Female	8-19-1994
17	0021-IKXGC	vishakha	India	Rajasthan	Female	9-2-2000
18	0022-TCJCI	raghvendra	India	Telangana	Male	12-30-1983
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	5-14-1991
20	0023-UYUPN	sudevi	India	Maharashtra	Female	10-6-1977

In [113...

df_customers[['country' , 'state']]

Out[113...

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	None	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	None	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	None	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

In [114...

```
# country and state - unique value pair  
state_country_mapping = df_customers.dropna(subset=['country']).set_index('state',  
df_customers['country'] = df_customers['country'].fillna(df_customers['state'].m
```

In [115...

```
df_customers[['state', 'country']]
```

Out[115...

	state	country
0	Maharashtra	India
1	Karnataka	India
2	Delhi	India
3	Nagaland	India
4	Delhi	India
5	Delhi	India
6	Meghalaya	India
7	Rajasthan	India
8	Kathmandu	Nepal
9	Kathmandu	Nepal
10	Maharashtra	India
11	Karnataka	India
12	Telangana	India
13	Meghalaya	India
14	Uttar Pradesh	India
15	Delhi	India
16	Meghalaya	India
17	Rajasthan	India
18	Telangana	India
19	Uttar Pradesh	India
20	Maharashtra	India

In [116...

```
df_subscription.head()
```

Out[116...

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

In [117...

```
df_subscription.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    object
1   subscription_start_date 21 non-null    object
2   subscription_type      21 non-null    object
3   renewal_date          21 non-null    object
4   plan_type             21 non-null    object
5   contract_type         21 non-null    object
6   cancellation_date      6 non-null     object
7   cancellation_reason    6 non-null     object
8   monthly_charges       21 non-null    float64
9   cltv                  21 non-null    int64
10  churn_score           21 non-null    int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB

```

```

In [118... date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_subscription[date_col] = df_subscription[date_col].apply(pd.to_datetime)
df_subscription.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    object
1   subscription_start_date 21 non-null    datetime64[ns]
2   subscription_type      21 non-null    object
3   renewal_date          21 non-null    datetime64[ns]
4   plan_type             21 non-null    object
5   contract_type         21 non-null    object
6   cancellation_date      6 non-null     datetime64[ns]
7   cancellation_reason    6 non-null     object
8   monthly_charges       21 non-null    float64
9   cltv                  21 non-null    int64
10  churn_score           21 non-null    int64
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB

```

```

In [119... df_support.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     object
1   complaint_date  9 non-null     object
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     object
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes

```

```
In [120... df_support.drop(columns = ['col_1' , 'comment'] , inplace = True)
df_support.head()
```

```
Out[120...      customerid  complaint_date  escalations  csat_score
0    0003-MKNFE      8/28/2024           N         60
1    0003-MKNFE      8/28/2024           Y         10
2    0013-EXCHZ      1/20/2024           Y         20
3    0013-MHZWF      3/18/2025           N         90
4    0013-SMEOE      11/1/2024           N         30
```

```
In [121... # 3. Feauture Engineering and Data Analysis
```

```
In [122... df_subscription.head(3)
```

```
Out[122...      customerid  subscription_start_date  subscription_type  renewal_date  plan_type  cont
0    0002-ORFBO      2021-03-15           Refferal      2025-03-15    Standard
1    0003-MKNFE      2020-08-01           Paid      2024-08-01    Premium
2    0004-TLHLJ      2022-11-20           Organic      2025-11-20     Basic
```

```
In [123... # create a new col using existing col - churn flag
df_subscription['churn_flag'] = np.where(df_subscription['cancellation_date'].notna(), 1, 0)
df_subscription.head()
```

```
Out[123...      customerid  subscription_start_date  subscription_type  renewal_date  plan_type  cont
0    0002-ORFBO      2021-03-15           Refferal      2025-03-15    Standard
1    0003-MKNFE      2020-08-01           Paid      2024-08-01    Premium
2    0004-TLHLJ      2022-11-20           Organic      2025-11-20     Basic
3    0011-IGKFF      2019-05-10           Paid      2025-05-10    Premium
4    0013-EXCHZ      2023-01-05           Refferal      2024-01-05    Standard
```

```
In [124... df = (df_subscription
      .merge(df_customers , on = 'customerid' , how = 'left')
      .merge(df_support , on = 'customerid' , how = 'left'))
```

```
In [125... df.shape
```

```
Out[125... (23, 20)
```

```
In [126... df_subscription['customerid'].nunique()
```

```
Out[126... 21
```

```
In [127... df_customers['customerid'].nunique()
```

```
Out[127... 21
```

```
In [128... df_support['customerid'].nunique()
```

```
Out[128... 7
```

```
In [129... df_support['customerid'].size
```

```
Out[129... 9
```

```
In [130... df_support['complaint_count'] = df_support.groupby('customerid')['customerid'].t
```

```
In [131... df_support
```

```
Out[131...      customerid  complaint_date  escalations  csat_score  complaint_count
```

0	0003-MKNFE	8/28/2024	N	60	2
1	0003-MKNFE	8/28/2024	Y	10	2
2	0013-EXCHZ	1/20/2024	Y	20	1
3	0013-MHZWF	3/18/2025	N	90	1
4	0013-SMEOE	11/1/2024	N	30	1
5	0017-IUDMW	4/10/2024	Y	25	1
6	0019-EFAEP	9/27/2024	Y	30	1
7	0022-TCJCI	9/13/2024	Y	10	2
8	0022-TCJCI	9/14/2024	N	90	2

```
In [132... df_support = df_support.sort_values('complaint_date').drop_duplicates('customerid')
```

```
In [133... df_support
```

```
Out[133...      customerid  complaint_date  escalations  csat_score  complaint_count
```

2	0013-EXCHZ	1/20/2024	Y	20	1
4	0013-SMEOE	11/1/2024	N	30	1
3	0013-MHZWF	3/18/2025	N	90	1
5	0017-IUDMW	4/10/2024	Y	25	1
1	0003-MKNFE	8/28/2024	Y	10	2
8	0022-TCJCI	9/14/2024	N	90	2
6	0019-EFAEP	9/27/2024	Y	30	1

```
In [134... df_support['customerid'].size
```

```
Out[134... 7
```

```
In [135... # merge df
df = (df_subscription
      .merge(df_customers , on = 'customerid' , how = 'left')
      .merge(df_support , on = 'customerid' , how = 'left') )
```

```
In [136... df.shape
```

```
Out[136... (21, 21)
```

```
In [137... df.to_csv('exposed_churn_data.csv' , index = False)
```

```
In [ ]:
```

```
In [ ]:
```

```
In [138... # data A
```

```
In [ ]:
```

```
In [139... df = pd.read_csv("exposed_churn_data.csv")
print(df)
```


	customerid	subscription_start_date	subscription_type	renewal_date	\
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	
10	0017-DINOC	2021-12-01	Organic	2025-12-01	
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	
12	0018-NYROU	2020-10-09	Organic	2025-10-09	
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	
15	0020-INWCK	2023-05-06	Organic	2025-05-06	
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	
17	0021-IXXGC	2021-07-07	Paid	2025-07-07	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	

	plan_type	contract_type	cancellation_date	cancellation_reason	\
0	Standard	Annual	NaN	NaN	
1	Premium	Annual	2024-09-10	Switched to competitor	
2	Basic	Monthly	NaN	NaN	
3	Premium	Annual	NaN	NaN	
4	Standard	Monthly	2024-02-28	Too expensive	
5	Standard	Annual	NaN	NaN	
6	Basic	Monthly	2024-11-15	Not enough content	
7	Premium	Annual	NaN	NaN	
8	Standard	Monthly	NaN	NaN	
9	Basic	Annual	NaN	NaN	
10	Premium	Annual	NaN	NaN	
11	Standard	Monthly	2024-05-01	Poor streaming quality	
12	Standard	Annual	NaN	NaN	
13	Basic	Monthly	2024-10-31	Switched to competitor	
14	Premium	Annual	NaN	NaN	
15	Standard	Monthly	NaN	NaN	
16	Premium	Annual	NaN	NaN	
17	Standard	Annual	NaN	NaN	
18	Basic	Monthly	2024-09-14	Forgot to cancel trial	
19	Premium	Annual	NaN	NaN	
20	Standard	Monthly	NaN	NaN	

	monthly_charges	cltv	...	churn_flag	customer_name	country	\
0	13.99	627	...	0	keshav	India	
1	12.99	1150	...	1	raghav	India	
2	6.99	210	...	0	lalita	India	
3	22.99	1725	...	0	mohan	India	
4	13.99	195	...	1	mira	India	
5	17.99	720	...	0	durga	India	
6	8.99	230	...	1	mina	India	
7	22.99	1840	...	0	madan	India	
8	13.99	240	...	0	maya	Nepal	
9	6.99	335	...	0	arjun	Nepal	
10	22.99	1610	...	0	shiva	India	
11	7.99	270	...	1	rangadevi	India	
12	13.99	980	...	0	chitra	India	

13	12.99	160	...	1	raju	India
14	92.99	2185	...	0	Madhav	India
15	13.99	640	...	0	parvati	India
16	20.99	550	...	0	rikim	India
17	13.99	840	...	0	vishakha	India
18	16.99	42	...	1	raghvendra	India
19	22.99	1955	...	0	rishabh	India
20	13.99	790	...	0	sudevi	India

	state	gender	dob	complaint_date	escalations	csat_score	\
0	Maharashtra	Male	4-12-1982	NaN	NaN	NaN	
1	Karnataka	Male	11-23-1995	8/28/2024	Y	10.0	
2	Delhi	Female	2-15-1978	NaN	NaN	NaN	
3	Nagaland	Male	8-30-2001	NaN	NaN	NaN	
4	Delhi	Female	5-5-1990	1/20/2024	Y	20.0	
5	Delhi	Female	12-10-1988	3/18/2025	N	90.0	
6	Meghalaya	Female	9-21-1976	11/1/2024	N	30.0	
7	Rajasthan	Male	3-14-1999	NaN	NaN	NaN	
8	Kathmandu	Female	7-7-1985	NaN	NaN	NaN	
9	Kathmandu	Male	10-29-1993	NaN	NaN	NaN	
10	Maharashtra	Male	1-22-1997	NaN	NaN	NaN	
11	Karnataka	Female	6-18-1981	4/10/2024	Y	25.0	
12	Telangana	Female	12-1-2004	NaN	NaN	NaN	
13	Meghalaya	Female	4-25-1992	9/27/2024	Y	30.0	
14	Uttar Pradesh	Male	11-11-1979	NaN	NaN	NaN	
15	Delhi	Female	2-28-1986	NaN	NaN	NaN	
16	Meghalaya	Female	8-19-1994	NaN	NaN	NaN	
17	Rajasthan	Female	9-2-2000	NaN	NaN	NaN	
18	Telangana	Male	12-30-1983	9/14/2024	N	90.0	
19	Uttar Pradesh	Male	5-14-1991	NaN	NaN	NaN	
20	Maharashtra	Female	10-6-1977	NaN	NaN	NaN	

	complaint_count
0	NaN
1	2.0
2	NaN
3	NaN
4	1.0
5	1.0
6	1.0
7	NaN
8	NaN
9	NaN
10	NaN
11	1.0
12	NaN
13	1.0
14	NaN
15	NaN
16	NaN
17	NaN
18	2.0
19	NaN
20	NaN

[21 rows x 21 columns]

In [140... df.columns

```
Out[140...] Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='object')
```

```
In [141...] churn_rate = df['churn_flag'].mean()*100
print("churn Rate = " , round(churn_rate , 2), "%" )
```

churn Rate = 28.57 %

```
In [142...] # 2. Retention Rate
Retention_rate = 100 - churn_rate
print("Retention Rate = " , round(Retention_rate , 2), "%")
```

Retention Rate = 71.43 %

```
In [143...] # 3. churn by Plan type
churn_by_plan =( df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).

print('churn by plan = ' , churn_by_plan)
```

churn by plan =	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [144...] # 4. Churn by state + sum(revenue) & count oof users
# 4. Churn by subscription type
# 4 . claculate customer age
```

```
In [145...] # 5. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu , 2) )
```

ARPU = 18.85

```
In [146...] df['subscription_start_date'] = pd.to_datetime(
      df['subscription_start_date']
    )

df['cancellation_date'] = pd.to_datetime(
      df['cancellation_date']
    )
```

```
In [147...] # 6. Avg Customer Tenure
# count of days users has used our service : cancellation date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
      df['cancellation_date'].notna(),

      (df['cancellation_date'] - df['subscription_start_date']).dt.days,
      (today - df['subscription_start_date']).dt.days
    )
avg_tenure = df['tenure_days'].mean()
print('Average Tenure = ', round(avg_tenure) , 0)
```

Average Tenure = 1529 0

```
In [148... # Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']== 1 , 'monthly_charges'].sum()
print("Revenue at Risk : Rs = " , revenue_at_risk,"k")
```

Revenue at Risk : Rs = 73.94 k

```
In [149... # 8. Escalation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ", round(escalation_rate , 2), "%")
```

Escalation Rate = 19.05 %

```
In [150... # 9. Avg complaint Per User
avg_complaints = df['complaint_count'].sum()/df['customerid'].nunique()
print("Avg Complaint Per User = ", round(avg_complaints , 2))
```

Avg Complaint Per User = 0.43

```
In [151... print(df['churn_flag'].unique())
print(df['escalations'].dtype)
```

[0 1]
object

```
In [152... print(df['churn_flag'].value_counts(dropna=False))
print(df['escalations'].unique())
```

churn_flag
0 15
1 6
Name: count, dtype: int64
[nan 'Y' 'N']

```
In [153... # 10. Correlation: Escalations vs Churn

corr_df = df[['escalations', 'churn_flag']].dropna().copy()

# Convert Y/N to 1/0
corr_df['escalations'] = corr_df['escalations'].map({
    'Y': 1,
    'N': 0
})

corr_df['churn_flag'] = corr_df['churn_flag'].map({
    'Y': 1,
    'N': 0
})

# Remove any NaN created during mapping
corr_df = corr_df.dropna()

# Calculate correlation
correlation = corr_df['escalations'].corr(corr_df['churn_flag'])

print("Correlation between escalation vs churn =", float(round(correlation, 2)))
```

Correlation between escalation vs churn = nan

```
In [49]: # 11. churn risk - create a column using existing columns
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
```

```

(df['churn_score'] >= 70)
]
choices = ['low' , 'med' , 'high']
df['churn_risk'] = np.select(conditions , choices , default = 'unknown')

```

In [50]: `df[['churn_score' , 'churn_risk']].head()`

Out[50]:

	churn_score	churn_risk
0	12	low
1	91	high
2	34	low
3	8	low
4	88	high

In []:

In []:

In []:

In [50]: *# 4. data Visualisation using matplotlib*

In [51]: `df_visual = df.copy()`

In [52]: `df_visual.columns`

Out[52]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'churn_risk'], dtype='object')

In [53]: *# 4.1 Monthly churn Trend (Time series KPI)*
#df_visual['cancellation_date'].dt.to_period('M')

```

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

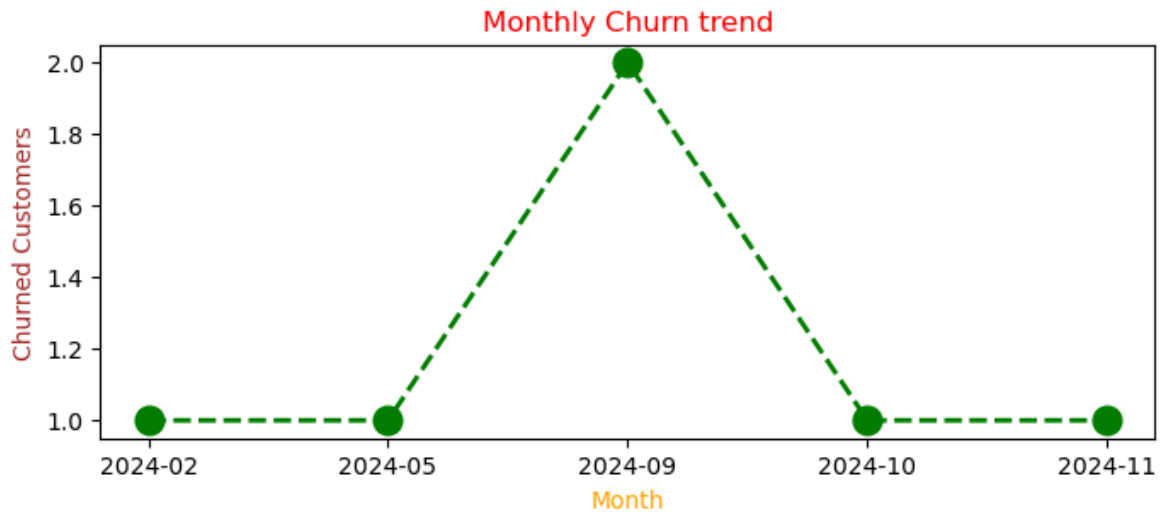
churn_trend = (
    df_visual[df_visual['churn_flag'] == 1]
    .groupby('cancellation_month')
    .size()
)

#print(churn_trend)
plt.figure(figsize = (8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values,color = 'green' , mar

plt.title('Monthly Churn trend' , color= 'red')

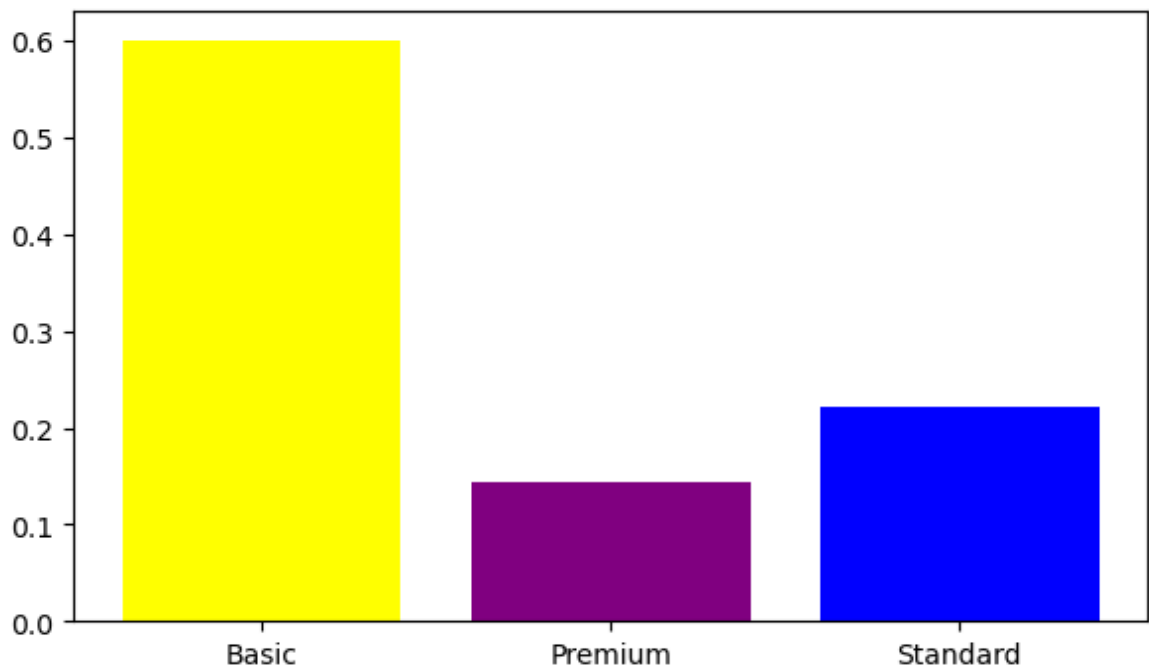
```

```
plt.xlabel('Month' , color = 'orange' )
plt.ylabel('Churned Customers' , color = 'brown')
plt.show()
```



```
In [54]: # 4.2 churn by plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

colors = ['yellow' , 'purple' , 'blue']
plt.figure(figsize = (7, 4))
plt.bar(churn_plan.index , churn_plan.values , color = colors)
plt.show()
```

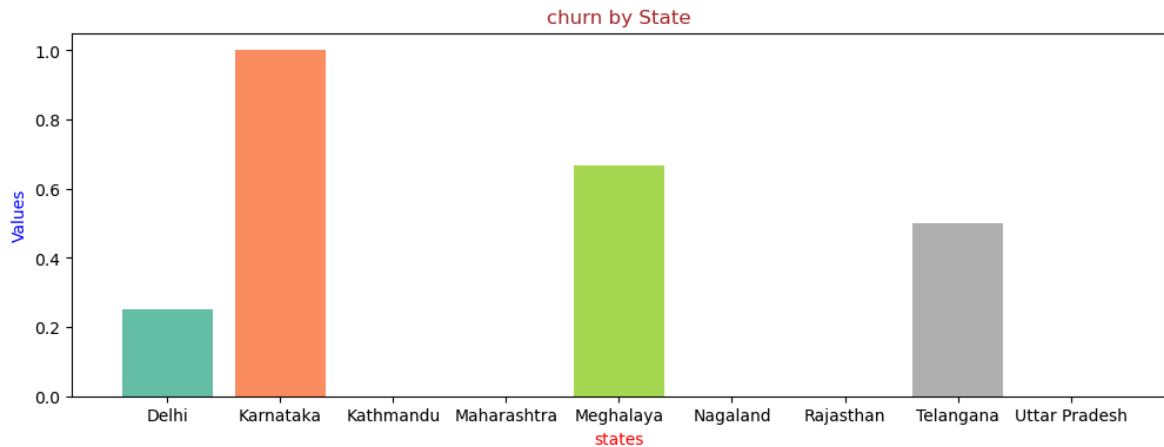


```
In [55]: #4.3 . churn by states
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

#pick the random color for the bars
color_s = plt.cm.Set2(np.linspace(0 , 1 , len(churn_plan)))

plt.figure(figsize = (12, 4))
plt.bar(churn_plan.index , churn_plan.values , color = color_s)
plt.title('churn by State' , color = 'brown')
plt.xlabel('states' , color = 'red')
```

```
plt.ylabel('Values' , color = 'blue')
plt.show()
```



```
In [94]: # visualisation using seaborn
```

```
In [56]: # encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

```
Out[56]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count',
               'tenure_days', 'churn_risk', 'cancellation_month'],
              dtype='object')
```

```
In [57]: df_visual[['plan_type' , 'contract_type' , 'churn_score' , 'churn_flag' , 'churn
```

```
Out[57]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	NaN
1	Premium	Annual	91	1	high	Y
2	Basic	Monthly	34	0	low	NaN
3	Premium	Annual	8	0	low	NaN
4	Standard	Monthly	88	1	high	Y

```
In [73]: import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# =====
# 1. Select required columns
# =====

df_encoded = df_visual[
    [
        'plan_type',
```

```

        'contract_type',
        'churn_score',
        'churn_flag',
        'churn_risk',
        'escalations'
    ]
].copy()

# =====
# 2. Convert categorical columns to numbers
# =====

# plan_type
df_encoded['plan_type'] = (
    df_encoded['plan_type']
    .astype('category')
    .cat.codes
)

# contract_type
df_encoded['contract_type'] = (
    df_encoded['contract_type']
    .astype('category')
    .cat.codes
)

# churn_risk
df_encoded['churn_risk'] = (
    df_encoded['churn_risk']
    .astype('category')
    .cat.codes
)

# =====
# 3. Convert churn_flag Y/N to 1/0
# =====

df_encoded['churn_flag'] = (
    df_encoded['churn_flag']
    .astype(str)
    .str.strip()
    .str.upper()
)

df_encoded['churn_flag'] = df_encoded['churn_flag'].replace({
    'Y': 1,
    'N': 0,
    'YES': 1,
    'NO': 0
})

df_encoded['churn_flag'] = pd.to_numeric(
    df_encoded['churn_flag'],
    errors='coerce'
)

# =====

```



```

# 4. Convert numerical columns
# =====

df_encoded['churn_score'] = pd.to_numeric(
    df_encoded['churn_score'],
    errors='coerce'
)

df_encoded['escalations'] = pd.to_numeric(
    df_encoded['escalations'],
    errors='coerce'
)

# =====
# 5. Fill missing values
# =====

for col in df_encoded.columns:
    if df_encoded[col].isnull().any():
        df_encoded[col] = df_encoded[col].fillna(
            df_encoded[col].median()
        )

# =====
# 6. Create correlation matrix
# =====

corr = df_encoded.corr()

# =====
# 7. Create heatmap
# =====

plt.figure(figsize=(8, 6))

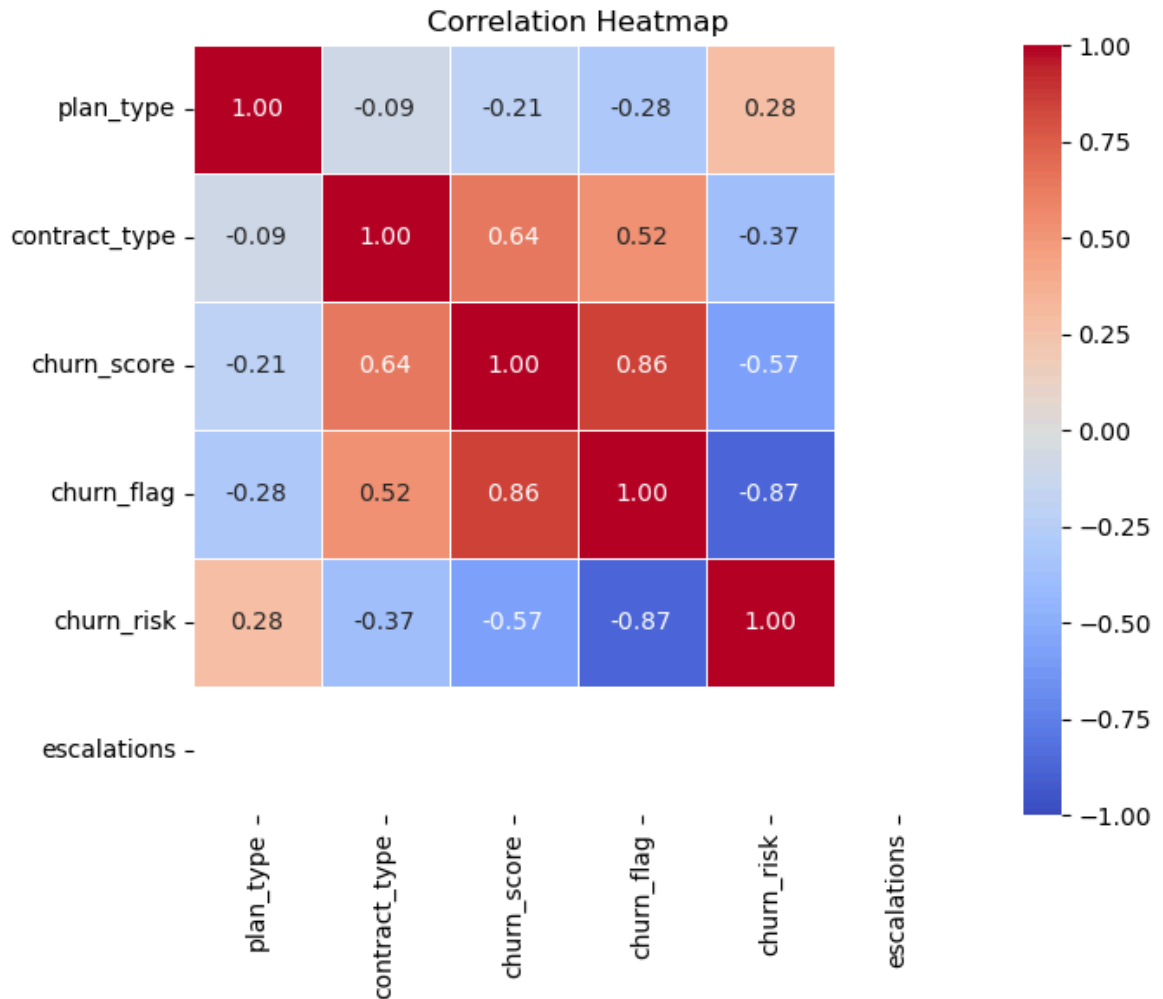
sns.heatmap(
    corr,
    annot=True,          # Show correlation values
    fmt='.2f',           # Show 2 decimal places
    cmap='coolwarm',     # Same type of colors
    vmin=-1,
    vmax=1,
    square=True,
    linewidths=0.5,
    cbar=True
)

plt.title('Correlation Heatmap')

plt.tight_layout()

plt.show()

```



```
In [76]: # correct method of encoding - based on priority
df_encoded = df_visual[['plan_type' , 'contract_type' , 'churn_score' , 'churn_f

order_mappings = {
    'plan_type' : ['Basic' , 'Standard' , 'Premium'],
    'contract_type' : ['Monthly' , 'Annual'],
    'churn_risk' : ['low' , 'med' , 'high']
}
for col , order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categori
```

```
In [77]: df_encoded.head()
```

```
Out[77]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	NaN
1	2	1	91	1	2	Y
2	0	0	34	0	0	NaN
3	2	1	8	0	0	NaN
4	1	0	88	1	2	Y

```
In [78]: df_visual[['plan_type' , 'contract_type' , 'churn_score' , 'churn_flag' , 'churn
```

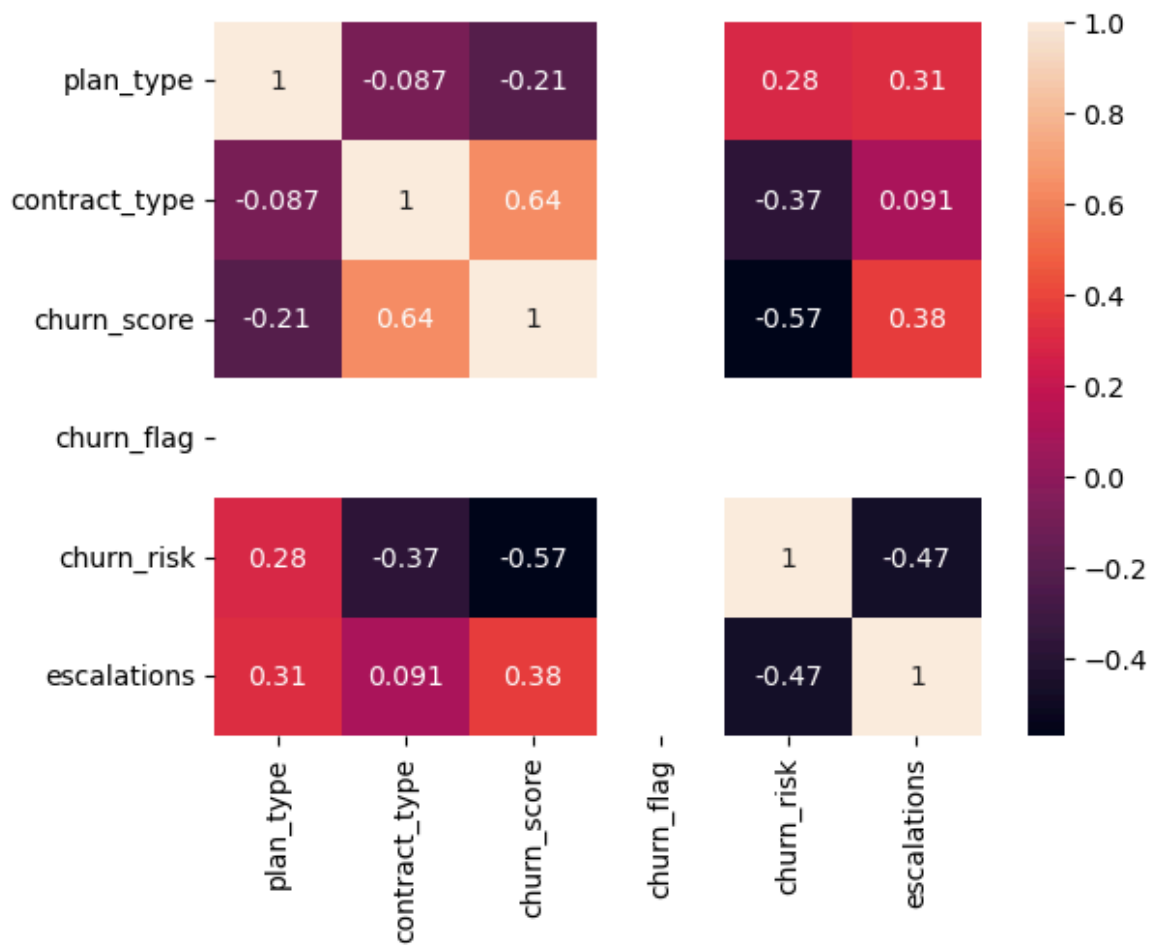
```
Out[78]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	NaN
1	Premium	Annual	91	1	high	Y
2	Basic	Monthly	34	0	low	NaN
3	Premium	Annual	8	0	low	NaN
4	Standard	Monthly	88	1	high	Y

```
In [89]: df['escalations'] = df['escalations'].map({'Y': 1, 'N': 0})
```

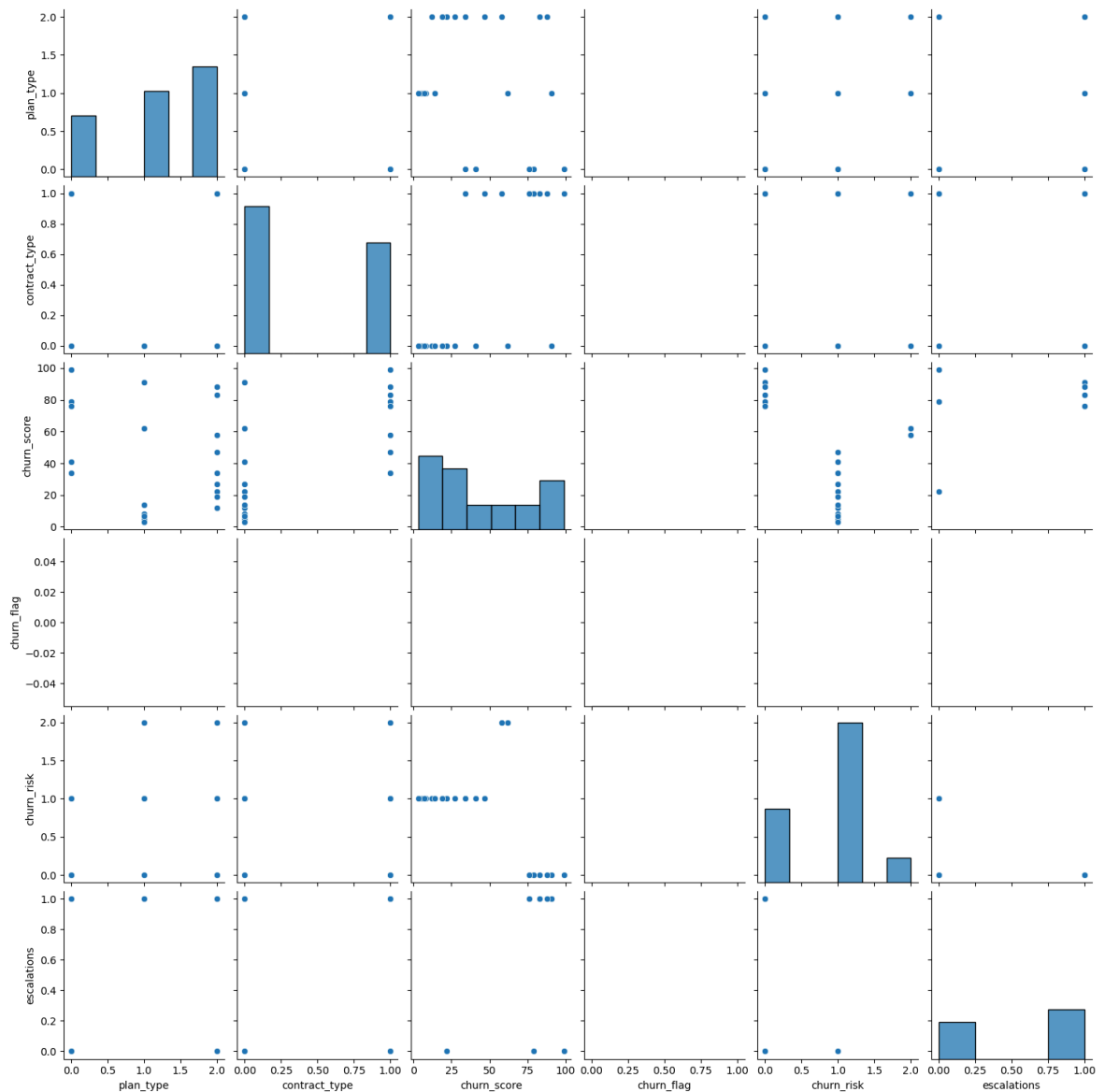
```
In [90]: sns.heatmap(df_encoded.corr() , annot = True)
```

```
Out[90]: <Axes: >
```



```
In [91]: # Pairplot
sns.pairplot(df_encoded)
```

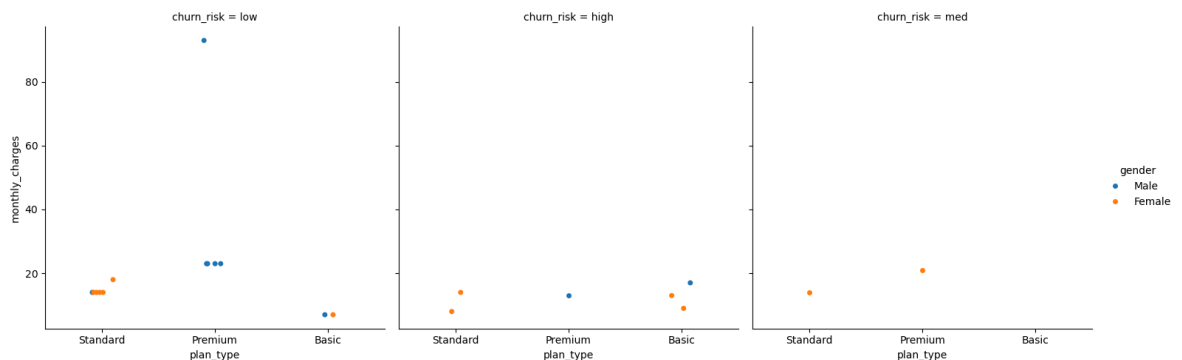
```
Out[91]: <seaborn.axisgrid.PairGrid at 0x28fea458ad0>
```



```
In [92]: # catplot/ Facegrid plot

sns.catplot(data = df_visual ,
            x = 'plan_type',
            y = 'monthly_charges',
            hue = 'gender' ,
            col = 'churn_risk')
```

Out[92]: <seaborn.axisgrid.FacetGrid at 0x28fea4586e0>



```
In [ ]: # Pivot talbes
```

```
In [154... pd.pivot_table(
    df_visual ,
    values = 'churn_flag' ,
    index = 'plan_type',
    aggfunc = 'mean'

).reset_index()
```

```
Out[154...    plan_type  churn_flag
0      Basic    0.600000
1    Premium    0.142857
2    Standard    0.222222
```

```
In [156... pd.pivot_table(
    df_visual ,
    index = 'plan_type',
    values = ['monthly_charges' , 'customerid' , 'churn_flag'] ,
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique' ,
        'churn_flag' : 'mean'
    }

)
```

```
Out[156...    churn_flag  customerid  monthly_charges
plan_type
Basic    0.600000         5         52.95
Premium  0.142857         7        218.93
Standard 0.222222         9        123.91
```

```
In [157... # Working with SQL in Python (Pandas)
```

```
In [161... # create db in sql
conn = sqlite3.connect('test_database.sqlite')

# table details
conn.execute("CREATE TABLE users (first_name TEXT , country TEXT , budget INTEGE

# commit and save
conn.commit()
```

```
In [169... # insert data

cursor = conn.cursor()

cursor.execute(
    """
    INSERT INTO users VALUES
        ('Madhav' , 'India' , 5000),
        ('Rishabh' , 'Germany' , '2500'),
```

```

        ('Vishakha' , 'India' , 3500)

        """
    )

    # commit and save
    conn.commit()

    print("Data Insert successfully")

```

Data Insert successfully

```

In [170... # Check insert data in table

conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query , conn)

df_results.head()

```

```

Out[170...
   first_name  country  budget
0    Madhav    india    5000
1    Rishabh  Germany    2500
2   Vishakha    India    3500
3    Madhav    india    5000
4    Rishabh  Germany    2500

```

```

In [171... # aggregation

query = """
        SELECT country , SUM(budget) as total_budget
        FROM users
        GROUP BY country

        """

df_agg = pd.read_sql(query , conn)
df_agg

```

```

Out[171...
   country  total_budget
0  Germany         10000
1    India         24000
2   india         10000

```

```

In [172... # Always close conn with db once the task is over
conn.close()

```

```

In [ ]:

```